

Fundamentals of Programming & Computer Science CS 15-112

Strings

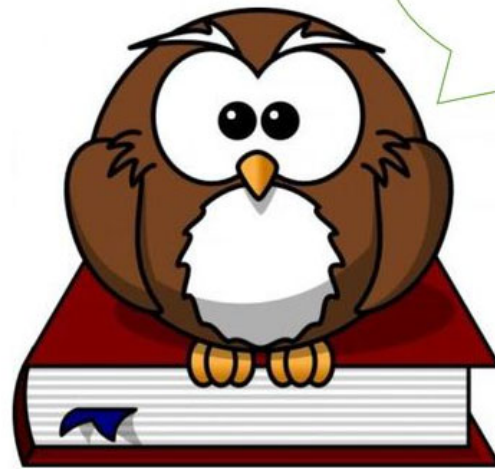
Dr. Hend Gedawy

Last Lecture

- **Strings**

- Strings defined with “ ” or ‘ ’
- `len(s)` : give length in a string
- Indexing:
 - `s[i]`, `s[-i]`
- Slicing, examples:
 - `s[i:n]`, `s[:n]`, `s[i:]`
 - `s[::step]`, `s[::-1]`
 - `S[i:s][i:s]`
- Strings are immutable
- Accessing the chars in a string
 - For `c` in `s`:
 - For `i` in `range(len(s))`: ...





**Let's
practice!**

```

1 def f1(s):
2     return s[::2][::-1] + s[::-1][::2]
3
4 def f2(s):
5     s = s[: len(s) // 2] + s[len(s) // 4 : len(s) - 1]
6     return s
7
8 def ct1(magic):
9     print("1: ", f1(magic))
10    print("2: ", f2(magic))
11    return magic
12
13 print(ct1("112rocks"))

```

Strings Practice

- Write the Function `nonVowelsCount(s)` that takes a string `s` and returns number of non-vowels inside `s`. Ignore case, A and a are both vowels.

```
1 def ct2(s):
2     result = ""
3     i = 0
4     for c in s:
5         if c.isdigit():
6             result += s[int(c)]
7         elif c == c.upper():
8             i += 3
9         else:
10            result += s[i % 2]
11            i += 1
12    return result
13
14 s = "x3Y5cA"
15 print(ct2(s))
```

String Practice

Write the function `numberSum(s)` that returns the sum of all numbers in a string

If the digits are next to each other, you have to treat them as a holistic number

```
numberSum("A2bru500RG") # returns 502
```

Built-in String Functions

1. String Creation:

- `str()`: Converts an object into a string.

2. String Concatenation:

- `+`: Concatenates two strings.

3. String Length:

- `len()`: Returns the length of a string.

4. String Indexing:

- `[]`: Accesses individual characters in a string.

5. Substring Extraction:

- `str[start:end]`: Returns a substring from index `start` to `end-1`.
- `str[:end]`: Returns a substring from the beginning to index `end-1`.
- `str[start:]`: Returns a substring from index `start` to the end.

6. String Repetition:

- `*`: Repeats a string a specified number of times.

7. String Case Conversion:

- `str.upper()`: Converts all characters to uppercase.
- `str.lower()`: Converts all characters to lowercase.
- `str.capitalize()`: Capitalizes the first character.
- `str.title()`: Converts the first character of each word to uppercase.

<https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str>

8. String Stripping:

- `str.strip()`: Removes leading and trailing whitespaces.
- `str.lstrip()`: Removes leading whitespaces.
- `str.rstrip()`: Removes trailing whitespaces.

9. String Splitting:

- `str.split()`: Splits a string into a list of substrings based on whitespace.
- `str.split(sep)`: Splits a string into a list using the specified separator.

10. String Joining:

- `str.join(iterable)`: Concatenates strings from an iterable using the specified string as a separator.

11. String Formatting:

- `str.format()`: Formats a string by substituting placeholders with values.
- `f"{variable}"` (f-strings): Allows embedding expressions inside string literals.

12. String Search:

- `str.find(substring)`: Returns the lowest index of the substring. Returns -1 if not found.
- `str.index(substring)`: Returns the lowest index of the substring. Raises an exception if not found.
- `str.count(substring)`: Returns the number of occurrences of the substring.

13. String Replacement:

- `str.replace(old, new)`: Replaces occurrences of the old substring with the new substring.

14. String Checking:

- `str.startswith(prefix)`: Checks if the string starts with the specified prefix.
- `str.endswith(suffix)`: Checks if the string ends with the specified suffix.
- `str.isalpha()`: Checks if all characters in the string are alphabetic.
- `str.isdigit()`: Checks if all characters in the string are digits.
- `str.isalnum()`: Checks if all characters in the string are alphanumeric.
- `str.isspace()`: Checks if all characters in the string are whitespaces.

String Practice

6. Free Response: Word Builder

We say that a word, `w1`, can be used to build another word, `w2`, if some subset of the letters from `w1` can be rearranged to produce the word `w2`. For example, the word "water" can be used to build "rate", "tear", "at", etc. (But it can't be used to build "rater" because "water" only has one r, not two.)

(a) (8 points) Write the function `wordCanBeBuilt(s1, s2)` which returns `True` if the string `s1` can be used to build the string `s2`, and `False` otherwise.

(b) (7 points) Write the function `canBuild(word, manyWords)` that takes as arguments a string containing a word and another string containing a comma-separated list of words, and returns a comma-separated list of words from `manyWords` that can be built from `word`.

Consider the following test cases:

```
assert canBuild("damthewater", "ad,add,sad,ma,tear") == "ad,ma,tear"
assert canBuild("freakymonkeytips", "far,jet,pit,lip,jaw,sip") == "far,pit,sip"
bigWord = "supercalifragilisticexpialidocious"
bigList = "jet,pit,hero,sail,space,lack,pride,tile,park,drug,steam"
assert canBuild(bigWord, bigList) == "pit,sail,space,pride,tile,drug"
```

Notes:

- Remember that you can't use lists on this exam.
- `.split()` may be very useful here but you may only loop over the result, and may not index/slice the result or use list functions or list methods.
- You may assume that you have a version of `wordCanBeBuilt(s1, s2)` that works, even if yours from part (a) does not work.

String Practice

```
def ct(s):  
    r = ""  
    while (len(s) > 1):  
        r += s[:1] + s[2:4] + "-"  
        s = s[1:len(s)-1]  
        if s.isalpha():  
            s = s.upper()  
        print(s)  
    return r + s  
  
print(ct("abcd123"))
```

String Practice

Example: `leastFrequentLetters(s)`

- Write the function `leastFrequentLetters(s)`, that takes a string `s`, and ignoring case (so "A" and "a" are treated the same), returns a lowercase string containing the least-frequent alphabetic letters that occur in `s`.
- Each least-frequent letter is included only once in the result and in alphabetical order.
- Digits, punctuation, and whitespace are not letters!
- If `s` has no alphabetic characters, the result should be the empty string (`""`).
- Example:

```
leastFrequentLetters("aDq efQ? FB'daf!!!") == "be"
```